Parallel Computing Assignment 1

Implementation and experimental report | English edition: 24 October 2026
Reference experiments: 16 October 2026
Repository: GitGud031005/parallel-computing-assignment

Scope and deliverables

The implementation covers all three required shared-memory parts in C++20: TAS/TTAS/MCS mutex locks, a reusable Sense-Reversing Barrier, and concurrent counters using mutexes, compare-and-swap (CAS), and fetch-and-add (FAA). Standard mutex/barrier implementations provide comparison baselines. TTAS/CAS exponential backoff are additional experiments. Optional MPI-3 one-sided RMA is not implemented.

Requirement	Implementation / evidence
Locks and prime workload	TAS, TTAS, MCS, std::mutex, TTAS-backoff; identical naive is_prime and outside-work contract.
Reusable barrier	Sense reversal and std::barrier; global episodes per second without useful work between episodes.
Counters	Four mutex counters, CAS, FAA, CAS-backoff; increments return the previous value.
Correctness and scaling	Directed tests, SC reference variants, sanitizer runs, raw records, median/IQR plots and reproducible scripts.

Measurements come from the managed Linux environment available during development, not the student's personal laptop. Experiments on the actual shared-memory machines owned by the submitter, identity details, and LMS submission remain to be completed.

Run: reference-20261016. Measured kernel/build SHA-256:
1695153b0caf2d8bd2a316b757addac3387b618dc3a72fe8cf47a29b1832720c

1. Memory model and mutex locks

Cache coherence alone does not establish visibility across multiple variables. A C++ data race on ordinary shared data causes undefined behavior. A lock must establish both mutual exclusion and publication of the previous owner's payload. Unlock uses release semantics; the successful acquiring operation receives the corresponding publication. Relaxed polling is only a waiting optimization [2].

Lock	Mechanism and ordering
TAS	Repeated atomic_flag test_and_set(acquire); clear(release). Repeated central RMW traffic; no FIFO guarantee.
TTAS	Poll with relaxed loads while busy, then exchange(acquire); store(release) on unlock. Reduces RMW traffic during waiting, with contention still possible at release.
MCS	Stable per-worker queue nodes, atomic tail and local waiting flag. FIFO follows tail-exchange order. A preempted owner can block the queue.
std::mutex	Standard-library baseline using the same workload and timing harness. Its internal implementation need not be a spinlock.

MCS initializes its node before tail.exchange(acq_rel). An enqueued worker publishes itself through predecessor.next with a release store and waits using acquire loads. Its predecessor releases the payload by storing waiting=false with release semantics. Node/context lifetime extends through unlock; a context is not copied or moved while in use.

There are two publication paths. With a successor, the waiting flag performs the handoff. With an apparently empty queue, the owner uses strong CAS to reset tail from its own node to null with release semantics. The next acquiring tail exchange receives that publication.

If CAS fails, a successor may have exchanged tail without linking next yet. The owner must wait for that link and then hand off. Weak CAS could fail spuriously when no successor exists and cause an indefinite wait here. A separate expected pointer prevents CAS from overwriting the identity of the node being unlocked. A directed test holds the successor exactly in this enqueue gap.

Spinlocks are blocking synchronization: progress can depend on a particular owner running. FIFO enqueue order is not invocation timestamp order, and a passing stress run does not prove starvation freedom.

2. Sense reversal and concurrent counters

The barrier has a fixed positive participant count P, remaining=P and global sense=false. Each worker has a separate local sense for each barrier instance. An arrival flips local sense and performs remaining.fetch_sub(1, acq_rel). The last arrival resets remaining=P with a relaxed store, then publishes global sense with release. Other participants wait with acquire loads.

The acq_rel RMW chain accumulates the writes preceding each arrival. The last thread receives this chain and publishes it through sense; waiting threads acquire that publication. Reset is sequenced before the release of sense. A second generation cannot finish until every participant has entered it, preventing a delayed participant from missing its previous generation.

One thread_local boolean shared by all barrier instances would be incorrect when instance phase counts differ. Explicit per-worker/per-instance contexts avoid this. Visibility tests use a second barrier before the next round of writes so that the test itself has no data race. The baseline calls std::barrier::arrive_and_wait [3].

Counter	Linearization and ordering
Mutex TAS/TTAS/MCS/std	An ordinary uint64_t increment while holding the lock. Concurrent value() calls acquire the same lock.
CAS	A successful CAS changing x to x+1. Weak-CAS retries recompute desired from the updated expected value.
FAA	fetch_add(1) is one atomic RMW and returns the previous value. Separate load/store would lose updates.
CAS-backoff	The same CAS algorithm with deterministic 1..1024 pause steps after failure, reset after each operation.

CAS/FAA use relaxed ordering because the counter represents only a number; it does not publish other payload. Increment returns the old value. Validated total work prevents uint64_t wrap. Actual atomic lock-freedom is platform dependent; CAS retry is not a per-thread wait-free guarantee. Returned values [0,K) and final K are empirical oracles; linearization points supply the general operation argument.

3. Correctness and course alignment

Test group	Oracle and directed cases
Workload	Naive prime against an independent square-root oracle for N=0..10009; quotient/remainder work distribution including K<P; input overflow.
Locks	Non-overlapping occupancy; protected counts; a separate plain-payload visibility test without telemetry RMW that could hide ordering errors; RAII and multiple instances.
MCS	Controlled enqueue gap and FIFO order, empty-queue handoff and node reuse.
Barrier	P=1/2/3/4/5/8, five seeds, many phases, A-A-B across instances, visibility and a delayed final participant; reject P=0.
Counters	No lost updates, exact old-value range, concurrent reads respecting each thread's completed increments, post-join precedence and no-wrap boundaries.
CLI / analysis	Prime/composite checksums, non-divisible work, invalid inputs; known-answer throughput, IQR/speedup and corrupted-row rejection.

All six CTest groups use runtime checks in Release. SC reference implementations supplement acquire/release variants; stress increases work tenfold and includes bounded oversubscription. Test timeouts detect a failed run, not correctness.

Recorded Debug, Release, ASan+UBSan, UBSan and TSan runs passed, as did lock, barrier and counter stress checks. LeakSanitizer was explicitly disabled because this host blocks /proc/ptrace inspection; address and undefined-behavior checks remained enabled. This is not a leak-free certification.

The algorithms follow course material: TAS/TTAS/MCS in Ch03 pp.17-28, sense reversal in Ch04 pp.8-15, CAS/FAA in Ch02a pp.16-21, and C++ ordering in Ch02b. Spinlock progress and counter specifications follow Ch03/Ch05. The repository's docs/course-review.md records the complete mapping. Finite tests and sanitizers do not prove every interleaving.

4. Platform and measurement protocol

Property	Recorded value
Platform / OS	managed-linux-x86_64 / Linux-6.18.44-x86_64-with-glibc2.39
CPU	AMD EPYC 9V74 80-Core Processor
Resources	9 visible logical CPUs; affinity 9; cpu.max=800000 100000 (8 CPU quota); memory limit 8589934592 bytes.
Toolchain	GCC 13.3.0; Release -O3 -DNDEBUG; no LTO or sanitizer.
Atomics / waiting	uint64, bool and pointer atomics reported lock-free; x86 pause; 64-byte layout alignment.
Control	Inherited affinity; power mode and external host load uncontrolled.

Naive is_prime tries divisors 2..N-1 and returns early on a divisor. Each lock iteration acquires, calls it once, releases, then performs W outside calls. The function is in a separate translation unit with IPO/LTO disabled. Local checksums are consumed after join. Optimized disassembly confirmed external calls remain in the intended loops. volatile is not used as synchronization.

Program time starts immediately before thread spawn and ends after join, including the independent latch start gate. Context/work allocations precede this timer. Worker interval is max(end)-min(start) after the gate. The barrier executes E episodes with no useful work between; throughput is E/worker_seconds, not P*E/seconds.

The reference matrix has 395 configurations, two warmups and seven measured repetitions each: 3555 raw samples, zero errors. Configuration order uses fixed shuffle seed 20261016. Median and inclusive IQR are descriptive statistics, not confidence intervals.

Core locks use N=997, W=0/1/2/4/8 and P=1/2/3/4/8; fixed-total K=20000 or per-thread I=5000. Sensitivity uses N=97/K=200000 and N=10007/K=2000 with W=0/4. Barriers use E=20000; counters K=1000000. Short runs remain sensitive to startup and scheduling. Per-thread ratios are not strong-scaling speedups.

5. Mutex lock results

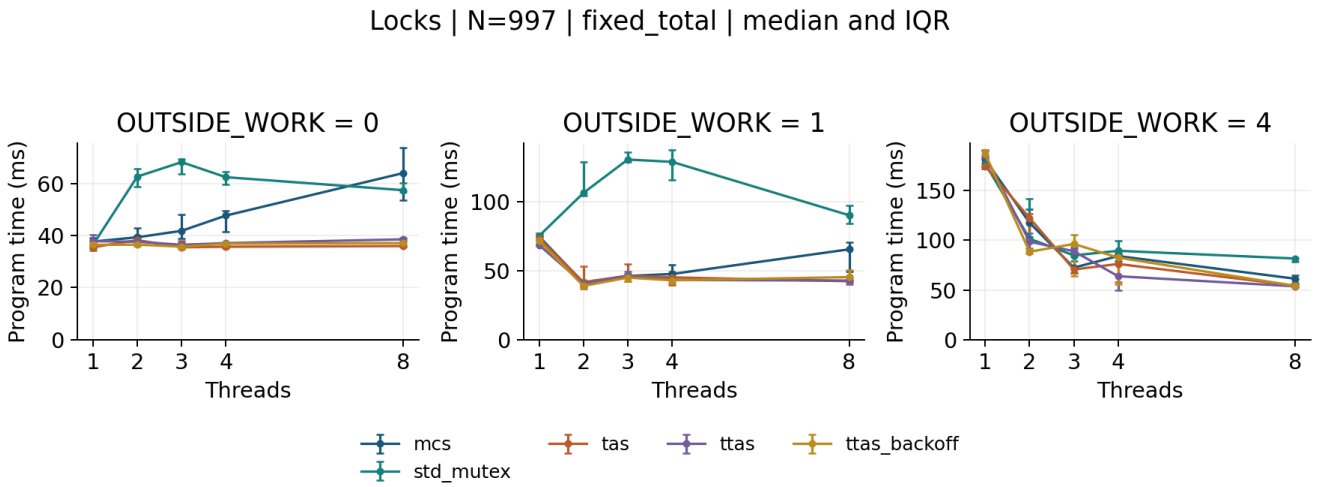


Figure 1. Fixed total K=20000, N=997; median and IQR. Each comparison holds K, N and W constant.

P=8, W=0	Median ms	IQR ms	S(P)
mcs	64.059	53.705-73.747	0.589
std_mutex	57.496	56.737-60.218	0.635
tas	35.972	35.610-36.321	0.987
ttas	38.563	37.870-39.253	0.982
ttas_backoff	37.139	35.891-38.318	0.981

TAS has the lowest median in this selected configuration. This is a workload/platform-specific observation, not a universal ranking. S(P)=T(1)/T(P) uses program time and the same implementation, K, N and W. Small percentage differences alone do not establish statistical significance.

Useful work inside the critical section remains serial. TAS repeatedly updates a shared flag; TTAS reads while busy; MCS replaces central flag contention with queued nodes and handoffs. These are algorithmic explanations, not measured cache-miss or NUMA attribution. A preempted owner and long critical sections can change the tradeoffs.

6. Outside work and critical-section sensitivity

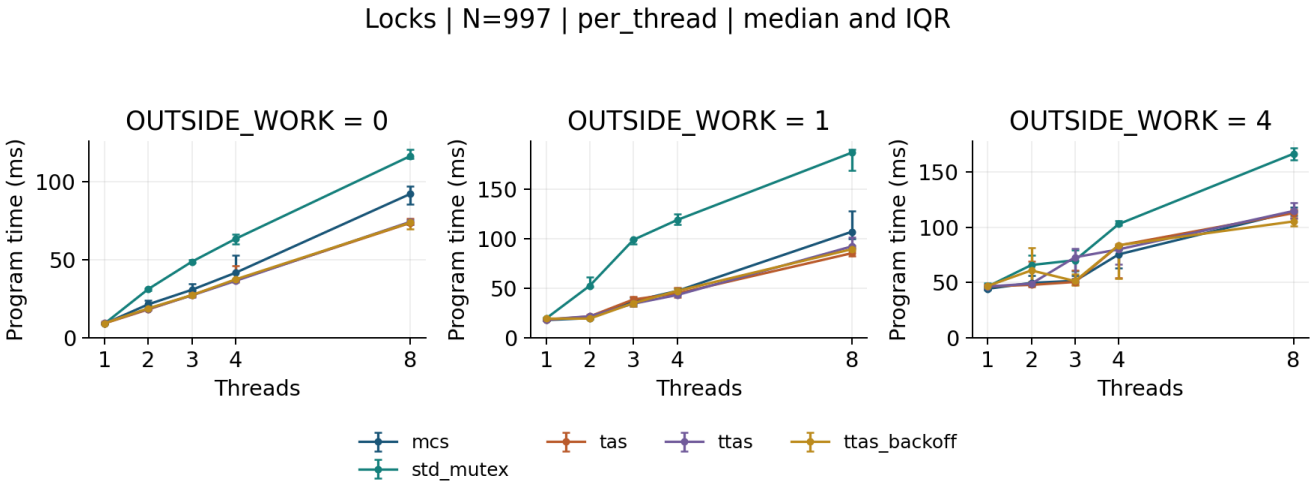


Figure 2. Per-thread I=5000. Total critical-section operations grow as P*I; these curves must not be interpreted as strong scaling.

OUTSIDE_WORK=k puts 1/(k+1) of prime calls inside the critical section. This is a fraction of calls, not elapsed time, because lock/spin/scheduler overhead also contributes. More outside work can reduce contention while increasing total computation. Every sample checks the expected total call checksum.

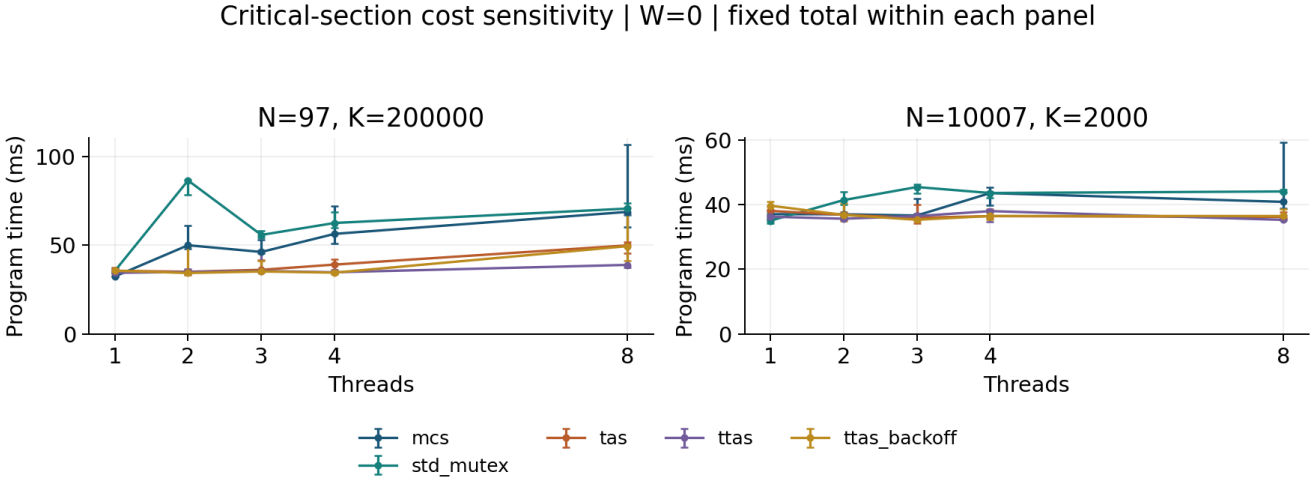


Figure 3. Short/long N sensitivity; K differs between panels to keep durations practical. Compare implementations and P only within a panel. Backoff uses 1..1024 pause instructions, not nanosecond sleeps. It can help or hurt depending on retry and waiting costs.

7. Barrier throughput

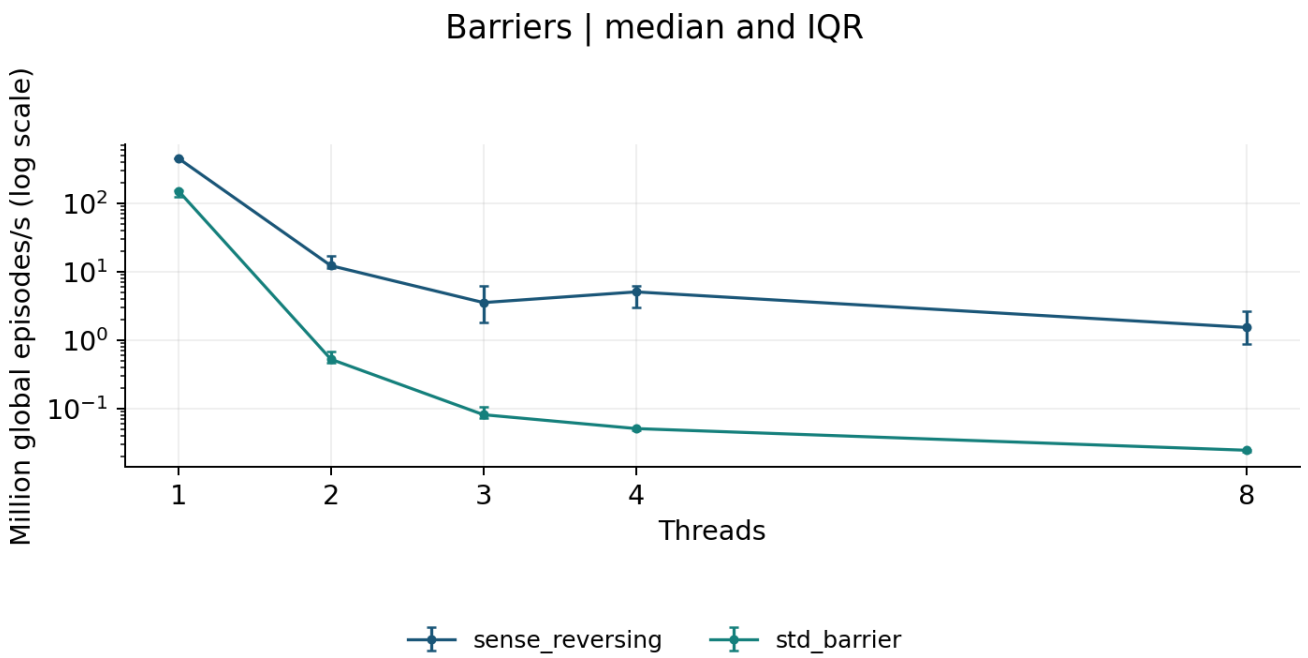


Figure 4. Global episodes per second, median and IQR, without useful work between barriers.

P=8	Median episodes/s	Latency/episode (us)
sense_reversing	1,539,853.092	0.649
std_barrier	24,843.989	40.251

Each of P workers makes E calls, but the program completes E global episodes. P*E/t measures arrival throughput, a different unit. Timers are read at worker boundaries, not every phase. The standard library and centralized custom barrier have different implementation/platform tradeoffs; the custom remaining/sense state is still shared contention.

A slow participant delays the whole phase. Busy waiting can be especially sensitive to preemption under oversubscription. The performance sweep stops at eight threads according to CPU quota. Oversubscription in correctness stress does not replace measurements on the submitter's actual machines.

8. Concurrent counter performance

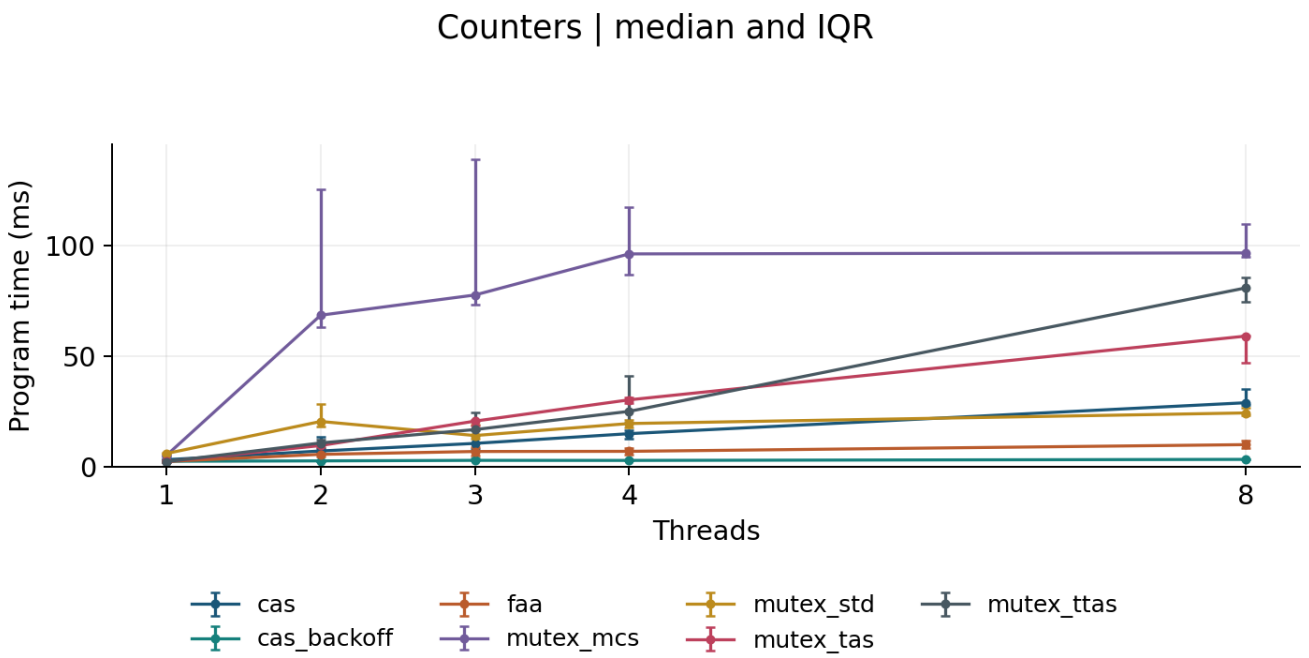


Figure 5. Fixed K=1000000 increments; spawn-to-join program time, median and IQR.

P=8	Median ms	Median increments/s
cas	29.086	34,380,911.690
cas_backoff	3.539	282,571,605.058
faa	10.190	98,132,094.828
mutex_mcs	96.645	10,347,146.881
mutex_std	24.488	40,836,244.617
mutex_tas	59.142	16,908,331.245
mutex_ttas	80.884	12,363,318.415

CAS-backoff has the lowest median in this table. All variants validate final K and the old-value checksum. CAS may retry; FAA directly expresses the increment RMW; mutex counters add lock/unlock. All still update one shared numeric location. More workers do not create independent useful counter work.

Backoff shifts retry timing and adds waiting. No diagnostic shared retry counter is added to the timed hot loop. Fast execution does not prove wait-freedom, fairness or starvation freedom.

9. Reproduction, limitations and references

Run from the repository root with CMake >=3.20, C++20 including std::barrier, and Python >=3.10. Python plotting/PDF dependencies are listed in assignments/assignment1/scripts/requirements.txt. Use a fresh output directory; runners reject overwriting previous data.

```
cmake -S . -B build/release -DCMAKE_BUILD_TYPE=Release
cmake --build build/release -j 4
ctest --test-dir build/release --output-on-failure
python3 assignments/assignment1/scripts/run_benchmarks.py \
--bin-dir build/release/assignments/assignment1 \
--matrix assignments/assignment1/config/benchmark_matrix.json \
--platform-id my-computer --output results-local/run-01
```

Then run summarize.py, plot_results.py, create_report.py and package_submission.py with the actual run/summary/figure/check paths documented in README. Keep checks.json and source fingerprints. Extract the submission ZIP into a fresh directory and build/test again.

This English edition translates the recorded reference study; it does not claim new benchmark runs. The measured Ass1 fingerprint predates integration of Ass2 into root CMake. Current integrated Ass1 fingerprint is 582f0be6e5f8aca6b8d197c0fc62b078c7d98e0a7dc23e011f9c24302311ea3f. The lock/barrier/counter algorithms were unchanged; the historical measurements retain their original fingerprint.

Limitations: one virtualized managed platform, uncontrolled external load/power, no personal-machine or ARM results, short-run startup effects, no hardware counters or exhaustive model checking. Complete identity details, check the LMS closing time, and run on owned platforms before final submission. The supplied statement specifies 29 October 2026.

References

[1] Assignment1_v1_16092026.pdf, pp.1-4, supplied course material.
[2] Course slides Ch00-Ch07 (including Ch02a/Ch02b), 234 PDF pages; Ch03 pp.17-28, Ch04 pp.8-15. Page numbers are one-based PDF pages.
[3] C++20 draft N4861: atomics.order, atomics.types.operations, thread.barrier.class. https://timsong-cpp.github.io/cppwp/n4861/
[4] Mellor-Crummey and Scott, Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors, ACM TOCS (1991). https://www.cs.rochester.edu/research/synchronization/pseudocode/ss.html